

Phishing. Técnicas y métodos de ataque y cómo detectarlos

Sistema cliente-servidor con análisis explicable y modelos centralizados

Alejandro Villarrubia García

Tutor: Carmelo González García · UEMC

TFG

El problema exige una decisión explicable

¿Cómo detectar phishing sin perder la explicación?

Una etiqueta aislada no basta para revisar un correo sospechoso.

01

Señales técnicas

Resultados SPF, DKIM y DMARC presentes en cabeceras, dominios, URLs, HTML y adjuntos.

02

Ingeniería social

Urgencia, autoridad, credenciales y patrones BEC.

03

Decisión trazable

Riesgo, señales activadas y explicación para el usuario.

Objetivos del prototipo desarrollado

Una única lógica de análisis; varias formas de usarla.

El cliente recoge datos y presenta resultados. El servidor concentra parser, detectores, entrenamiento y versiones.

01

Aceptar varias entradas

Texto, EML, Gmail, extensión y monitor.

02

Combinar dos enfoques

Heurística explicable + TF-IDF y MLP bilingüe.

03

Compartir los modelos

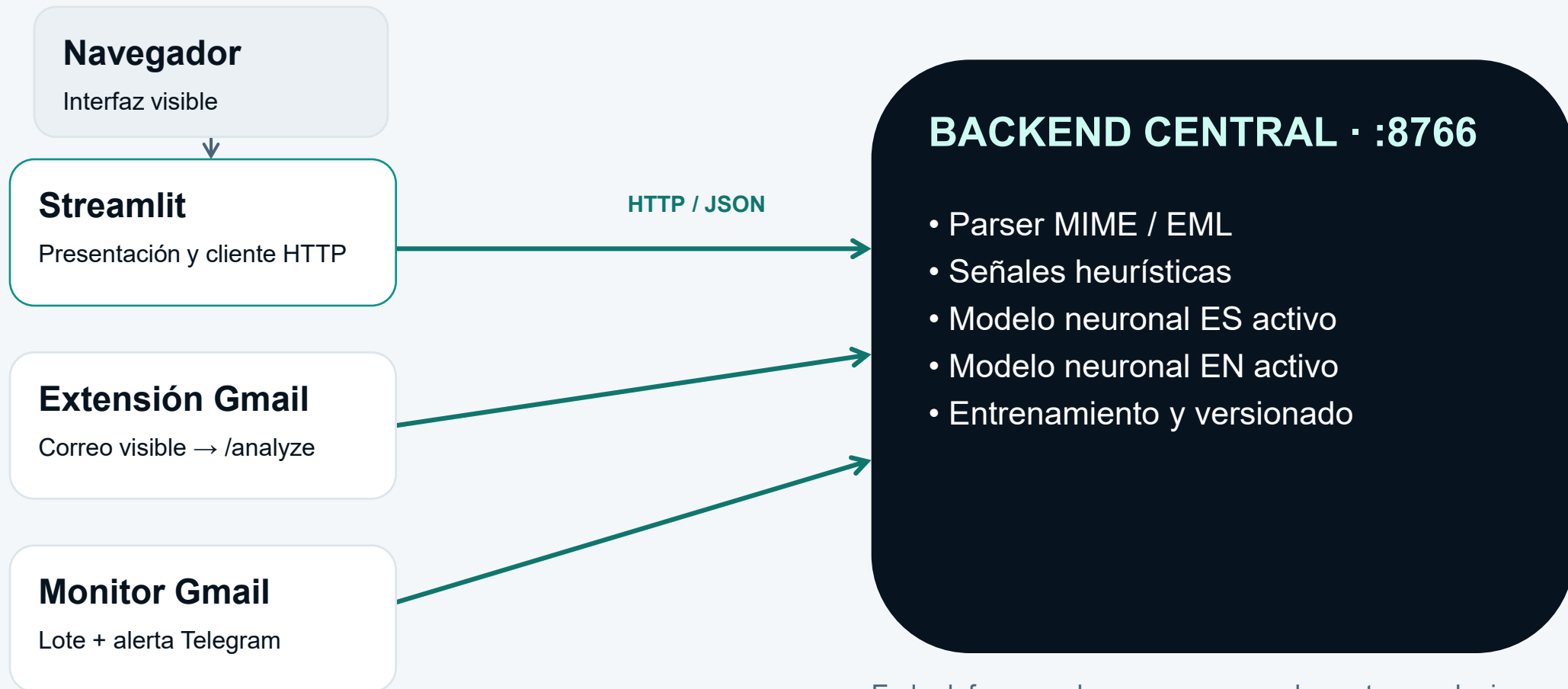
Una versión activa ES/EN para todos los clientes.

04

Evaluar con trazabilidad

Calibración separada, EML reservados y pruebas reproducibles.

Arquitectura cliente-servidor



En la defensa ambos procesos pueden estar en el mismo equipo; siguen separados por un contrato HTTP.

Cada correo sigue un flujo único y trazable



El contrato /analyze limita la entrada, devuelve una respuesta común y mantiene la lógica fuera de los clientes.

Reglas y clasificador neuronal

HEURÍSTICA EXPLICABLE

Inspecciona indicios concretos

Lectura pasiva de SPF/DKIM/DMARC en cabeceras

Remitente, dominio y URLs

HTML, adjuntos y redirecciones

Urgencia, credenciales y BEC

CLASIFICADOR NEURONAL

Aprende patrones del lenguaje

TF-IDF con n-gramas

Perceptrón multicapa (MLP)

Modelos separados ES / EN

Puntuación de clase positiva

CALIBRACIÓN LOCAL

45/55 · 40 casos · umbral 21 · alta confianza ≥ 70

Una versión activa sirve a todos los clientes

01 · ENTRENAR

02 · ACTIVAR

03 · COMPARTIR

El cliente envía CSV e hiperparámetros

El backend crea un pipeline nuevo desde cero.

Guardado atómico y caché invalidada

Una versión activa por idioma: ES y EN.

Web, extensión y monitor

La siguiente petición usa la versión nueva.

Resultado: cambiar el modelo en el servidor actualiza a todos sin reinstalar clientes.

La tecnología prioriza claridad y control

DECISIONES

Python

Núcleo, API, pruebas y automatización.

Streamlit

Cliente web rápido y adaptable.

scikit-learn

Pipeline reproducible TF-IDF + MLP.

Gmail + Telegram

Entrada OAuth y alertas opcionales.

CONTROLES DEL PROTOTIPO

- Backend en loopback por defecto
- Límites de petición y rutas seguras
- Orígenes CORS restringidos
- Credenciales fuera de Git
- Escrituras y activación atómicas
- Sin ejecución de adjuntos

Un despliegue público requeriría TLS, autenticación y aislamiento multiusuario.

La evaluación separa ajuste y comprobación

1.148 + 209

65.661 + 16.416

40 + 16 EML



Corpus español

Entrenamiento limpio y split oficial de prueba sin solapamientos.

Corpus inglés

División 80/20 estratificada tras deduplicar el CSV agregado.

Sistema completo

Calibración separada y comprobación bilingüe con MIME y cabeceras.

Principio metodológico: ninguna muestra se presenta como estimación de producción.

El heurístico obtiene el mejor resultado en 16 EML

Evaluación final · 16 EML reservados



100 % recall

Combinado: 8/8 phishing
2 falsas alarmas

Diagnóstico DIFrauD

89,0 % accuracy

93,4 % recall

Diagnóstico complementario; no acredita superioridad en producción.

La evidencia cubre código, web y rendimiento

94

pruebas Python

Componentes, API, modelos e integraciones

2

recorridos Chromium

Incluido navegador → Streamlit → backend

96,6 %

menos tiempo de importación

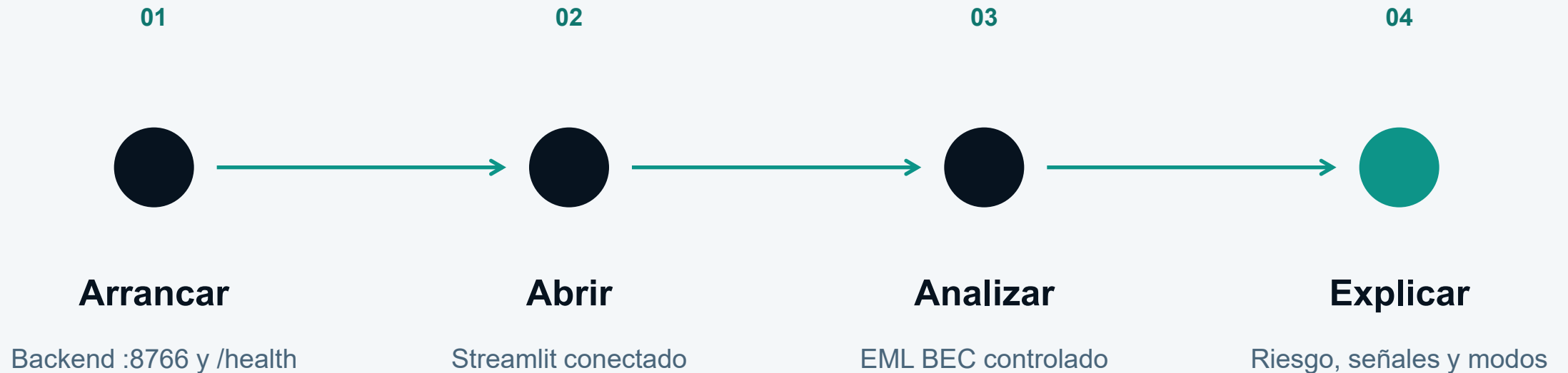
Heurísticas tras diferir dependencias pesadas

76,2 %

menos arranque de la web

Mejora medida; inferencia estable

Cuatro pasos sostienen la demostración



Opcional: móvil en la LAN → Streamlit 0.0.0.0:8501; el backend permanece en 127.0.0.1:8766.

Los límites están definidos, no ocultos

LO QUE DEMUESTRA

- Arquitectura cliente-servidor funcional
- Análisis explicable y bilingüe
- Modelos centrales compartidos
- Evaluación reproducible y errores trazables

LO QUE NO AFIRMA

- Métricas garantizadas en producción
- Reputación online ni validación TLS
- Análisis dinámico de las páginas
- Seguridad multiusuario para Internet

PRIORIDAD FUTURA

Corpus reciente, bilingüe, anonimizado e independiente · TLS y autenticación · reputación de dominios

Una decisión central, varias interfaces

El prototipo separa presentación y análisis, combina evidencias explicables con aprendizaje supervisado y permite actualizar los modelos una sola vez para todos los clientes.

Aportación: un detector mantenible, evaluado y preparado para evolucionar sin duplicar modelos.

Preguntas del tribunal